

Curso:

FUNDAMENTOS DE LA PROGRAMACIÓN

MATERIAL DE CONSULTA COMPLEMENTARIO 2

LÓGICA COMPUTACIONAL



Bloques temáticos

1. Lógica Matemática
2. Proposiciones
3. Operaciones lógicas
4. Algebra
 - 4.1. Expresiones y variables
 - 4.2. Expresiones con dos variables
 - 4.3. Ejemplos
5. Sistemas de numeración
 - 5.1. Sistemas de numeración posicional
 - 5.2. Relaciones entre conjuntos de tipos numéricos
6. Representaciones de números en distintas bases
 - 6.1. Conversión de cualquier base a base 10
 - 6.2. Conversión de decimal a otras bases
 - 6.3. Casos particulares
7. Matrices

ANEXO 1 - PREGUNTAS FRECUENTES

1. Lógica Matemática

La lógica matemática es una variedad de la lógica filosófica, permite el análisis del razonamiento, trata el estudio y análisis de los métodos de razonamiento o argumentación. La lógica centra su análisis en la relación entre los enunciados, pero no en el contenido de los mismos, es decir no nos permite determinar si los enunciados son verdaderos o falsos, pero partiendo de su verdad o falsedad nos permite determinar si las conclusiones son verdaderas o falsas.

Ejemplo:

- **Enunciado 1:** Todos los alumnos de Matemática recuerdan los conceptos adquiridos en la secundaria.
- **Enunciado 2:** En la secundaria se adquirió el concepto de conjunto.
- **Por lo tanto:** Todos los alumnos de Matemática comprenden el concepto de conjunto.

Si consideramos que los enunciados son verdaderos, la conclusión es verdadera.



La lógica no se centra en el análisis de los enunciados (no sabe si son verdaderos o falsos) pero partiendo de su afirmación o negación podemos decir si la conclusión es correcta o incorrecta (verdadera o falsa).

Los objetivos principales de la lógica son esencialmente:

- Eliminar las ambigüedades propias del lenguaje ordinario.
- Dar rigor a aquello que se está estudiando.

¿Por qué es esto importante en la programación? Porque las operaciones lógicas son parte fundamental de todo desarrollo de software, ya que constantemente tenemos que decidir qué hacer ante diferentes estímulos que recibe nuestro programa. Para esto será importante comprender como plantearnos preguntas en forma abstracta para luego llevarlas a un lenguaje de programación para crear nuestro software.

2. Proposiciones

Como se vio en el ejemplo del punto anterior, el razonamiento parte de lo que llamamos “**enunciados**”. Estos enunciados **son** la base de la lógica y reciben el nombre de **proposiciones**.

Por lo general **son afirmaciones** y no instrucciones o preguntas (que es justamente lo que hacemos cuando programamos: **indicar acciones y hacernos preguntas**), **y pueden ser verdaderas o falsas, pero no tener una ambigüedad**.

Proposición:



Se entiende como proposición o enunciado a una oración declarativa sin ambigüedad, que es verdadera (V) o falsa (F), pero nunca las dos cosas simultáneamente.

La veracidad o falsedad de un enunciado se llama su “**Valor de Verdad**”. Los términos verdadero o falso se consideran como **atributos** de una proposición.

Por lo general se utilizan letras minúsculas para representar las **proposiciones** (Ej.: **p, q, r, s, t, etc.**).

Consideramos **proposiciones validas aquellas que carecen de ambigüedad** (son verdaderas o falsas, pero no ambas).

Vemos algunos ejemplos de proposiciones válidas y no válidas, y se explica por qué algunos enunciados no son proposiciones. Las proposiciones se indican por medio de una letra minúscula, dos puntos y la proposición propiamente dicha.

Ejemplos:

p: La luna es un planeta.

q: $(17)^3 = 21$

r: $x = y + 12$

w: Hola ¿cómo estás?

t: Camina dos pasos hacia la derecha.

- **p** y **q** sabemos que pueden tomar un valor de falso o verdadero, pero no
- ambos, por lo tanto, son proposiciones válidas.
- **r** también es una proposición válida, aunque el valor de falso o verdadero depende del valor asignado a las variables **x** e **y** en determinado momento.
- **t** y **w** no son válidos, ya que no pueden tomar un valor de falso o verdadero si

Como ya dijimos la lógica centra su atención en la relación entre los enunciados (proposiciones) y no en el contenido de los mismos.

Las proposiciones resultado de las relaciones entre proposiciones simples, se denominan **proposiciones compuestas**.

3. Operaciones lógicas

Disyunción: Dadas dos proposiciones **p** y **q**,

Se llama disyunción de **p** y **q**, y se escribe:

$$p \vee q$$

Y se la lee como "**p o q**".

Definición: $p \vee q$ es verdadera cuando al menos una de ellas lo es. En otras palabras, solamente será falsa cuando ambas proposiciones sean falsas.

Ejemplo:

- **p**: Tengo un piloto para la lluvia
- **q**: El día esta radiante
- **p $\vee$ q** : "Salgo si" tengo un piloto para la lluvia o el día esta radiante

Conjunción: Dadas dos proposiciones **p** y **q**,

Se llama conjunción de **p** y **q**, y se escribe:

$$p \wedge q$$

Y se la lee como "**p y q**".

Definición: $p \wedge q$ es verdadera cuando **p** y **q** son verdaderas simultáneamente.

Ejemplo:

- **n**: Tengo un piloto para la lluvia
- **q**: El día esta radiante
- **$p \wedge q$** : “Salgo si” tengo un piloto para la lluvia y el día esta radiante

Negación: Dada una proposición p , se llama negación de p , y se escribe $\neg p$ a la afirmación, y se dice "no p ".

Definición: $\neg p$ es verdadera cuando p es falsa (y viceversa)

Ejemplo:

- **p**: Tengo un piloto para la lluvia
- **$\neg p$** : “No” tengo un piloto para la lluvia
- **q**: El día esta radiante
- **$\neg q$** : El día “No” esta radiante

4. Algebra

El álgebra tiene probablemente su origen más bien "formal" en el libro "Compendio de cálculo por compleción y comparación", del matemático del siglo 8 o 9 llamado Al-Juarismi, que vivió en una región cercana a la actual Bagdad. Si bien existen registros del año 2000 AC sobre trabajos en álgebra en Babilonia, se considera al documento de Al-Juarismi como la base del álgebra moderna y a su autor como el "padre del álgebra", aunque no es posible dejar de lado otras grandes contribuciones de matemáticos importantes, como Diofanto de Alejandría o Brahmagupta de India.

Podemos definir el Algebra como "recuperación / compleción", literalmente. El concepto de "compleción" se refiere al sentido de completo o completitud que pudiera tener algo.

El álgebra se basa en el planteo de soluciones para determinas incógnitas.

En este sentido, las incógnitas suelen representarse como letras, como, por ejemplo:

$$x + 4 = 5$$

es

En un sentido práctico, el uso de letras se trata de una convención, ya que la misma expresión tendría le mismo significado que las siguientes:

$$_ + 4 = 5$$

$$? + 4 = 5$$

Siendo estrictos, la forma de trabajo sería la misma en cualquiera de los casos, aunque, nuevamente, por convención (además de claridad, facilidad de comprensión, entre otros factores) se asume el uso de letras como estándar.

4.1. Expresiones y variables

Las variables son valores que pueden cambiar de un momento a otro, ya que valor depende de factores externos o internos que no pueden ser determinados exactamente en forma predeterminada.

Para ejemplificarlo, supongamos el caso de un mozo, al que se le paga por día una cantidad fija como sueldo y a lo que se suman las propinas que recibe.

Veamos el flujo de caja de este mozo de la semana pasada (trabaja de domingo a viernes):

Día	Sueldo	Variable	
		Propinas	
DOM	100	450	
LUN	100	50	
MAR	100	100	
MIE	100	100	
JUE	100	150	
VIE	100	250	

Podemos ver que, siendo el domingo el día en el que atiende a más comensales es cuando recibió la mayor cantidad de dinero en propinas. El lunes, día en que se acercan pocos clientes es cuando menos recibe, pero va aumentando en la semana a medida que se aproxima el fin de semana.

Pero existen casos particulares, por ejemplo, los domingos del Día de la Madre y el Día del Padre, cuando se duplica la cantidad de clientes y sus propinas aumentan. Y, por el contrario, están los días festivos y feriados, en los cuales el restaurante cierra sus puertas, por lo que recibe su sueldo, pero no obtiene propinas.

En este ejemplo podemos identificar que **el sueldo es una constante**, ya que a través del período de tiempo analizado su valor no cambia. Por el contrario, las propinas que obtiene fluctúan en el tiempo debido al día de la semana, si es un día conmemorativo o festivo, entre otros factores. **Por esto es que decimos que las propinas son variables**, dado que **(NO** se pueden determinar por adelantado**)** qué valor tendrán.

Estas variables son las que en álgebra se representan con letras, como vimos antes.

En la programación, el concepto de "variable" y "constante" es muy importante, por lo que resulta interesante que se comprenda el origen del dicho término y su aplicación práctica en álgebra, ya que la programación lo toma de allí.

Siguiendo con el ejemplo, podemos representar la situación económica del mozo a través de la siguiente expresión:

$$s = 100 + p$$

Donde s es el sueldo que recibe y p la cantidad de propinas en dinero. Esta expresión representa los ingresos diarios del mozo, y aplica para cualquier día de la semana.

Evaluemos a continuación el siguiente caso:

“Una sociedad de fomento está vendiendo una colecta de fondos a través de la venta de "rifas". El costo de cada número está dado por la expresión $2 + (3 \cdot x)$, donde x es la cantidad de números”.

Evaluemos ahora el costo de adquirir 4 rifas o 6 rifas. Para esto, deberemos sustituir la cantidad de números (x) por estas cantidades, comenzando por el primer caso.

$$x = 4$$

$$s = 2 + (3 * x)$$

$$s = 2 + (3 * 4)$$

$$s = 2 + (12)$$

$$s = 14$$

El segundo caso, comprando 6 rifas sería así:

$$x = 6$$

$$s = 2 + (3 * x)$$

$$s = 2 + (3 * 6)$$

$$s = 2 + (18)$$

$$s = 20$$

¡¡RECUERDEN!!

Es común encontrar que el signo “*” (multiplicación) no se agrega en forma explícita en las expresiones, por lo que:

$$4 * x$$

Es equivalente a expresar:

$$4x$$

Supongamos un caso más práctico, en el cual estamos leyendo un libro de origen norteamericano, en el cual nos encontramos con una situación en ^{la} ~~el~~ que los protagonistas se encuentran escalando una montaña y, en el texto, nos dice que la temperatura es de 41 grados Fahrenheit. Para el que no tienen en claro la conversión podría resultar frustrante no saber cuánto representan los grados Fahrenheit (f) en los grados Celsius (c) que nos resultan más naturales.

Esto se podría solucionar sabiendo que la expresión para convertir grados f a grados c es la siguiente:

$$c = (f - 32) * 5/9$$

Aplicamos la sustitución de variables en la expresión dada y hacemos las cuentas:

$$c = (41 - 32) * 5/9$$

$$c = (9) * 5/9$$

$$c = 5$$

Finalmente, llegamos a la conclusión que los protagonistas se encontraban en la montaña a 5° Celsius. Hacía bastante frío después de todo... 😊

4.2. Expresiones con dos variables

Supongamos que nos encontramos con la siguiente expresión:

$$a + b$$

Y que tenemos que evaluar dicha expresión para los valores

$$a = 2$$

$$b = 3$$

Lo que hacemos a continuación, es sustituir los valores y realizar los cálculos:

$$a + b$$

$$2 + 3$$

$$= 5$$

De esta forma estamos resolviendo una expresión con dos variables.

Supongamos otro ejemplo:

$$5x + 3y$$

Si evaluamos la expresión para los valores:

$$x = 3$$

$$y = -2$$

Reemplazamos y realizamos los cálculos:

$$5(3) + 3(-2)$$

(Recuerden que esta expresión es equivalente a $5 \cdot (3) + 3 \cdot (-2)$)

$$15 + (-6)$$

$$= 9$$

4.3. Ejemplos

- a) Evaluar la expresión $a^2 + 5b - 6$ con $a = 2$ y $b = -3$
- b) Evaluar la expresión $3n^1 - 4n^0$ con $n=5$
- c) Evalúa la expresión $6a + 3b - 5$ con $a = 7$ y $b = 8$
- d) La suma de -4 y la cantidad de 3 veces x
- e) Toma la cantidad -5 veces x, y entonces suma uno
- f) -2 más el producto de -2 y x

a)

$$= 2^2 + 5(-3) - 6$$

$$= 4 + (-15) - 6$$

$$= 4 - 21$$

$$= - 17$$

b)

$$= 3n^1 - 4n^0$$

$$= 3(5^1) - 4(5^0)$$

$$= 3(5) - 4(1)$$

$$= 15 - 4$$

$$= 11$$

c)

$$= 6a + 3b - 5$$

$$= 6(7) + 3(8) - 5$$

$$= 42 + 24 - 5$$

$$= 61$$

d)

$$= -4 + 3x$$

e)

$$= -5x + 1$$

f)

$$= -2 + (-2x)$$

$$= -2 - 2x$$

5. Sistemas de numeración

La forma de escribir los números usada en la actualidad procede de sistemas de numeración de la India. Estos sistemas ofrecían dos ventajas sustanciales respecto a los usados en Europa:

1. El concepto del **número 0**, que, aunque probablemente fue importado de las culturas mesopotámicas, se integró por primera vez en un sistema decimal junto con las otras nueve cifras del sistema.
2. **Valor posicional a cada cifra**, de manera que (un mismo símbolo tiene un valor diferente según su posición) en la expresión numérica (compárenlo con el sistema de numeración Romano)

Este sistema fue adoptado por los árabes antes del siglo IX, y popularizado en el primer manual de aritmética inspirado en el sistema decimal posicional.

Sistemas de numeración:



Un sistema de numeración puede definirse como un conjunto de signos, relaciones, convenios y normas destinados a expresar de modo gráfico y verbal el valor de los números y las cantidades numéricas.

5.1. Sistemas de numeración posicional

En él se tienen en cuenta los siguientes aspectos:

- La **base del sistema** es la cantidad de símbolos con que se cuenta. Por ejemplo, la base 8 (octal) agrupa ocho símbolos, mientras que la base 10 (decimal) agrupa diez símbolos.

- **Los símbolos del sistema**, o **cifras** elementales que se utilizan. En el sistema decimal, se usan los símbolos 0, 1, 2, 3, 4, 5, 6, 7, 8 y 9. En cambio, por ejemplo, en el sistema hexadecimal se emplean el 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E y F.
- **Las normas de combinación** de los símbolos para formar los números. Según ello, a cada cifra ⁰~~0~~ símbolo se le asocian dos propiedades:
 - Su **valor absoluto intrínseco** y
 - Su **valor posicional o relativo**, que depende de la posición que ocupa
 - En la cantidad numérica.

Dado un número n escrito como la sucesión de numerales

$a_n, a_{n-1}, \dots, a_2, a_1, a_0$ en la base b

Puede descomponerse en forma polinómica del modo siguiente:

$$n_b = a_n, a_{n-1}, \dots, a_2, a_1, a_0$$

$$n_b = a_0 * b^0 + a_1 * b^1 + a_2 * b^2 + \dots + a_{n-1} * b^{n-1} + a_n * b^n$$

En esta forma polinómica se ve bien el **carácter posicional**, cada símbolo ($a_0, a_1, \dots, a_n$) esta multiplicado por un valor que depende de la posición en el número, este **número es la base elevada a la posición**:

Ejemplo:

A. $n = 134_8$

$$n = 1 * 8^2 + 3 * 8^1 + 4 * 8^0 = 92_{10} \quad (92_{10} = 134_8)$$

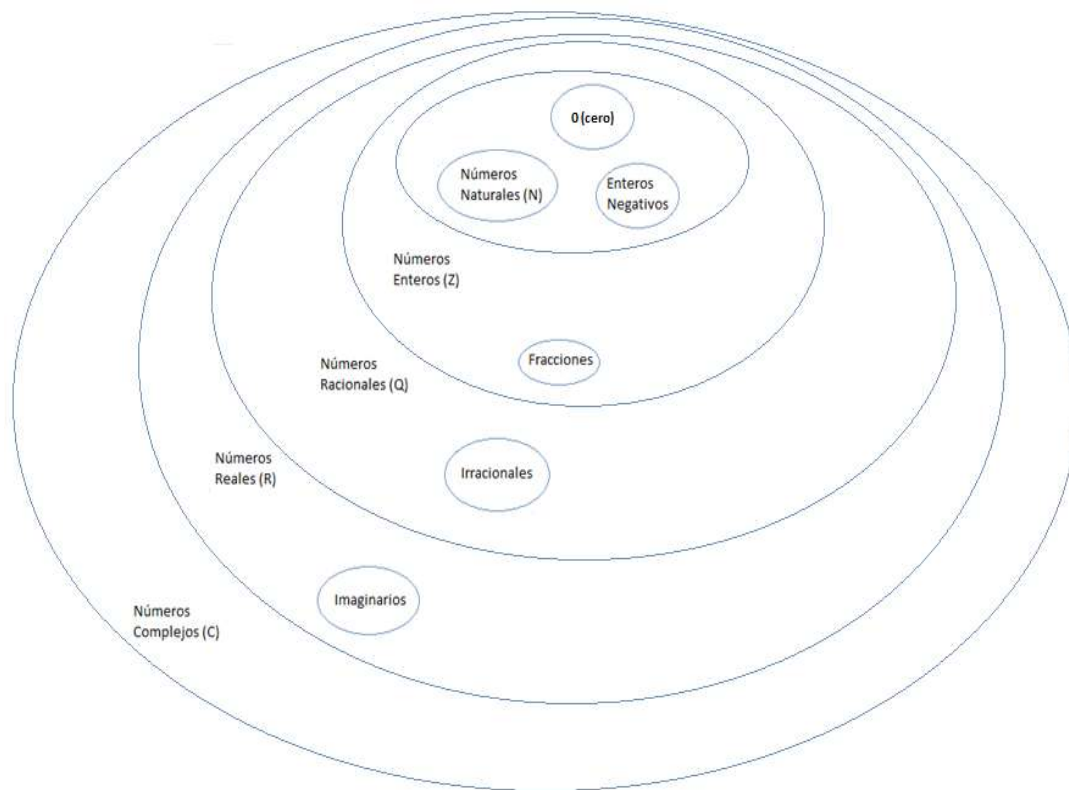
B. $n = 134_{10}$

$$n = 1 * 10^2 + 3 * 10^1 + 4 * 10^0 = 100 + 30 + 4 = 134$$

¿Qué nos dice el sistema de numeración posicional?

Si tenemos, por ejemplo, el número 321 y el número 213, podemos ver que ambos están compuestos por las mismas cifras: 1, 2 y 3. Entonces, si ambos tienen las mismas cifras, ¿podemos decir que es el mismo número? ¿321 y 213 son lo mismo? Evidentemente no. Esto es lo que nos dice la **numeración posicional**: un número tiene un significado (**valor**) que está **dada por** la ubicación (**orden**) **de las cifras** que lo componen. Es por eso que podemos distinguir 321 de 213.

5.2. Relaciones entre conjuntos de tipos numéricos



Este concepto de tipo de numéricos será importante que lo tengan presente para las próximas Unidades del curso donde será utilizados en conjunto con otros conceptos vistos aquí, aunque de momento es meramente informativo.

6. Representaciones de números en distintas bases

Como hemos visto, los números pueden representarse en sistemas de distintas bases, los cuales tienen una cantidad de símbolos igual al número de la base y el peso de la posición de esos símbolos depende también de la base del mismo.

La elección de **la base puede ser cualquiera y podemos convertir** la expresión de una cantidad **de una base a otra.**

En particular nos centraremos en algunas bases que son las más usadas:

- Base 10 (Decimal), cuyos símbolos son 0, 1, 2, 3, 4, 5, 6, 7, 8 y 9
- Base 2 (Binario), cuyos símbolos son 0 y 1
- Base 16 (Hexadecimal), cuyos símbolos son 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E y F

Veamos cómo es la notación del "treinta y cinco" en estos sistemas:

$$35_{10} \text{ }^{\circ} 100011_2 \text{ }^{\circ} 23_{16}$$

El símbolo " $^{\circ}$ " significa que dos valores son equivalentes, por lo que la primera expresión sería: "el número treinta y cinco en base decimal es equivalente al número uno, cero, cero, uno, uno en base binaria".

cero,

Veamos cómo se llega desde cada uno haciendo operaciones en base 10, **para** pasarlos a base decimal:

$$35_{10} = 3 * 10^1 + 5 * 10^0$$

$$= 3 * 10 + 5 * 1 = 35_{10}$$

NOTA: las potencias a las que se eleva cada término surgen de la posición en la que se encuentran dentro de la expresión, comenzando a contar desde 0 de derecha a izquierda. Por lo que en el número 35, el 5 correspondería a la posición 0, y ^{el} cálculo sería "5 * 10⁰". El 3 estaría en la posición 1, ~~por lo que se~~ "3 * 10¹", y así sucesivamente.

$$100011_2 = 1 * 2^5 + 0 * 2^4 + 0 * 2^3 + 0 * 2^2 + 1 * 2^1 + 1 * 2^0 =$$

$$= 1 * 32 + 0 * 16 + 0 * 8 + 0 * 4 + 1 * 2 + 1 * 1$$

$$= 32 + 2 + 1 = 35_{10}$$

Binario: ser o no ser:

Recurriendo a un viejo chiste (¿?), "hay 10 tipos de personas: los que entienden binario y los que no entienden binario", como es tan malo, hay que explicarlo: "10" en binario equivale a "2" en decimal.

Más allá de que hay métodos formales de conversión de bases, los primeros números en base decimal a binario se pueden obtener fácilmente, agregando dígitos binarios (usando solo 1 y 0) desde el número más chico posible.

En la primera columna tenemos los números en decimal y luego del "=" el número en binario.

0 = 0 (no hay duda, 0 es 0 en cualquier base)

1 = 1 (o sea, al 0 binario, le agrego un dígito binario)

2 = 10 (del 1 paso al 10, usando dígitos binarios (0's y 1's))

3 = 11 (el siguiente número a 10 usando 0's y 1's es 11)

4 = 100 (el número más chico que sigue a 11 binario es 100)

5 = 101 (siguiente número a 100 más chico que puedo armar con dígitos binarios)

6 = 110

7 = 111

8 = ...? Parecido al caso del "11" cuando tenemos que pasar al "100" Si: el 8 es 1000

9 = 1001 (¿el próximo más chico usando 1's y 0's desde el 1000 binario? Si, 1001)

10 = 1010

Y el resto es secuencial.

O sea, así como en decimal el número más chico que le sigue al 5 es 6, usando únicamente los dígitos binarios, para pasar del 101 el siguiente número binario es 110. Luego el 111. Luego el 1000, 1001, 1010, 1011, 1100, 1101, 1110, 1111, 10000, 10001, 10010, etc.

$$23_{16} = 2 * 16^1 + 3 * 16^0 = 2 * 16 + 3 * 1$$

$$= 32 + 3 = 35_{10}$$

Como vemos hemos utilizado la definición polinómica de un número.

$$n_b = a_n, a_{n-1}, \dots, a_2, a_1, a_0 = a_0 * b^0 + a_1 * b^1 + a_2 * b^2 + \dots + a_{n-1} * b^{n-1} + a_n * b^n$$

La conversión de la expresión de un número de cualquier base a base 10 se realiza teniendo en cuenta este polinomio y haciendo las operaciones en base 10.

6.1. Conversión de cualquier base a base 10

Ejemplos:

Hexadecimal → Decimal

$$\begin{aligned} A3_{16} &= A * 16^1 + 3 * 16^0 = 10 * 16 + 3 * 1 = \\ &= 160 + 3 = \\ &= 163_{10} \end{aligned}$$

Hexadecimal a decimal (ayuda extra)

Hay una relación directa entre los dígitos hexadecimales válidos con respecto a los números decimales:

- Del 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E y F en hexadecimal (son 16 números)

Tienen sus equivalentes en decimal:

- 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14 y 15

Por lo tanto, para convertir un número en hexadecimal a decimal:

$A3_{16} =$

Donde "A" en hexadecimal (viendo equivalencia comentada anteriormente) equivale al 10 en decimal

Donde 3 en hexadecimal equivale al 3 en decimal

Sabiendo esto, hacemos:

$$= A * 16^1 + 3 * 16^0$$

$$= 10 * 16^1 + 3 * 16^0$$

$$= (10 * 16) + (3 * 1)$$

$$= 160 + 3$$

$$= 163_{10}$$

Por lo tanto, A3 en hexadecimal equivale al 163 en decimal.

$$A3_{16} = 163_{10}$$

Otro ejemplo:

BDF_{16}

$$B_{16} = 11_{10}$$

$$D_{16} = 13_{10}$$

$$F_{16} = 15_{10}$$

Entonces:

$$= B * 16^2 + D * 16^1 + F * 16^0$$

$$= 11 * 16^2 + 13 * 16^1 + 15 * 16^0$$

$$= (11 * 256) + (13 * 16) + (15 * 1)$$

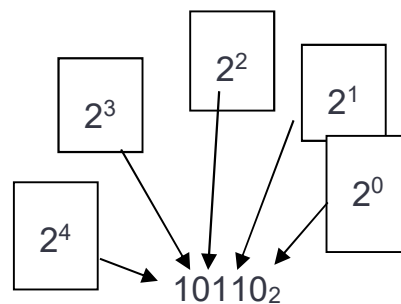
$$= (2816) + (208) + (15)$$

$$= 3039$$

Por lo tanto

$$BDF_{16} = 3039_{10}$$

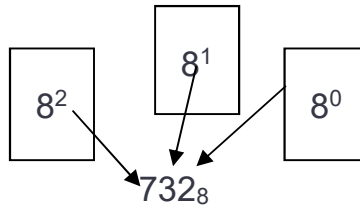
Binario → Decimal



$$10110_2 = 1 * 2^4 + 0 * 2^3 + 1 * 2^2 + 1 * 2^1 + 0 * 2^0 =$$

$$\begin{aligned}
 &= 1 * 16 + 0 * 8 + 1 * 4 + 1 * 2 + 0 * 1 = \\
 &= 16 + 0 + 4 + 2 + 0 = \\
 &= 22_{10}
 \end{aligned}$$

Octal → Decimal



$$\begin{aligned}
 732_8 &= 7 * 8^2 + 3 * 8^1 + 2 * 8^0 \\
 &= 7 * 64 + 3 * 8 + 2 * 1 = \\
 &= 448 + 24 + 2 = \\
 &= 474_{10}
 \end{aligned}$$

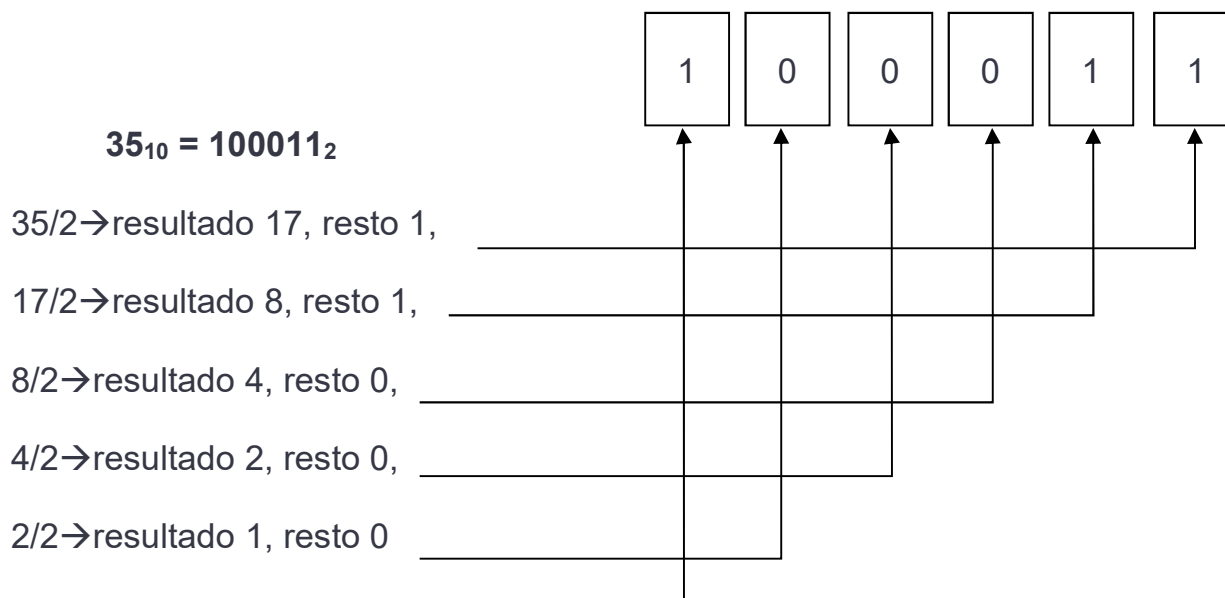
6.2. Conversión de decimal a otras bases

Algoritmo de la división

La conversión de un decimal a otras bases se realiza dividiendo el número decimal por la base a la que se quiere convertir, el resto de esa división será el dígito menos significativo del número en la nueva base. El resultado se vuelve a dividir por la base, el resto es el dígito siguiente en significatividad en la nueva base. Se continúa de esta manera hasta que el resultado es menor que la base, este resultado es el dígito más significativo en el número convertido. Veamos el mecanismo en práctica.

Ejemplos:

Decimal → Binario



Decimal → Hexadecimal



Decimal → Octal



6.3. Casos particulares

Conversión de octal a binario

Cada dígito se convierte separadamente a su equivalente en binario, expresado en 3 dígitos.

Ejemplo:

532₈ a binario

$$\left. \begin{array}{l} 5 = 101 \\ 3 = 011 \\ 2 = 010 \end{array} \right\} \quad 532_8 = 101011010_2 \quad (\text{Esto funciona porque } 8 \% 2 \text{ es } 0)$$

Conversión de binario a octal

(Esto funciona porque $\log_2(8)$ es 3)



Se separa el número binario en grupos de a tres dígitos de derecha a izquierda, cada uno de estos grupos se convertirá en su expresión en octal. Si el conjunto binario de la extrema izquierda no tiene 3 dígitos, agregar ceros hasta completarlo.

Ejemplo:

110111000100₂ a octal

$$\begin{array}{cccc} 110 & 111 & 000 & 100_2 \\ \hline 6 & 7 & 0 & 4_8 \end{array}$$

Conversión de hexadecimal a binario

Cada dígito se convierte separadamente a su equivalente en binario, expresado en 4 dígitos.

Ejemplo:

BC5₁₆ a binario

B(11) = 1011

C(12) = 1100

5 = 0101

BC5₁₆ = 101111000101₂ (Esto funciona porque 16 % 2 es 0)

Conversión de binario a hexadecimal

(Esto funciona porque $\log_2(16)$ es 4)

Se separa el número binario en grupos de a cuatro dígitos de derecha a izquierda, cada uno de estos grupos se convertirá en su expresión en hexadecimal. Si el conjunto binario de la extrema izquierda no tiene cuatro dígitos, agregar ceros hasta completarlo.

Ejemplo:

1111 1110 0010 1001 1110₂ a hexadecimal

1111 1110 0010 1001 1110₂

F E 2 9 E₁₆

¿Por qué son importantes estas representaciones?

- **Base 10:** Tomamos como punto de partir la base decimal, que es la que nos resulta más natural de entender a la gran mayoría.

- **Base 2:** A partir de ahí, vemos como se llega a convertir un número decimal a **binario**. Esto **es** relevante, ya que esta clase operaciones son las que hace y procesa, justamente, un procesador de una computadora. Pero no solamente un número es susceptible de ser convertido a decimal, sino también letras y símbolos que, en conjunto y agrupados, representan **todo aquello que realizamos en una computadora**.
- **Base 16:** Normalmente las direcciones de memoria RAM de una computadora (que tiene muchísimas combinaciones posibles), se representan con números hexadecimales ya que **se logra una importante abreviación** en cuanto a la cantidad de cifras utilizadas **con respecto a** números **decimales 0**, más aún, a los **binarios**.

Como conclusión, podemos ver que estas simples operaciones de transformación son TODO lo que ocurre dentro de una computadora, y esa es la misión y función del procesador: transmitir impulsos eléctricos que se expresan, en la teoría, como 1s y 0s, que es la base binaria y es la base a que se transforman todos los datos que maneja una computadora.

7. Matrices

Es posible que les resulte familiar las estructuras formadas por datos en filas y columnas como forma de almacenar información de manera ordenada. Un ejemplo de esto podría ser una planilla de cálculo, como las de Microsoft Excel o las de Google Spreadsheets.

Un ejemplo de datos almacenables en este tipo de estructuras es, por ejemplo, las posiciones de los clubes de fútbol en un campeonato, de modo que las filas dan información por cada equipo y las columnas lo hacen sobre los partidos jugados, ganados, empatados, perdidos, goles a favor, goles en contra y total de puntos. Como vemos a continuación en esta matriz compuesta de 21 filas y 9 columnas:

2014/2015								
PUN	EQUIPO	PJ	PG	PE	PP	GF	GC	PUN
1	Juventus	27	19	7	1	54	14	64
2	Roma	27	13	11	3	38	21	50
3	Lazio	27	15	4	8	49	27	49
4	Napoli	27	13	7	7	46	35	46
5	Fiorentina	27	12	9	6	39	29	45
6	Sampdoria	27	11	12	4	36	28	45
7	Internazionale	27	9	10	8	42	35	37
8	Genoa	26	9	10	7	37	32	37
9	Torino	27	9	9	9	30	30	36
10	Milan	27	8	11	8	38	34	35
11	Palermo	27	8	11	8	38	40	35
12	Udinese	26	8	8	10	29	34	32
13	Sassuolo	27	7	11	9	33	40	32
14	Hellas Verona	27	8	8	11	33	46	32
15	Empoli	27	5	15	7	27	29	30
16	Chievo	27	7	8	12	20	30	29
17	Atalanta	27	5	10	12	22	37	25
18	Cagliari	27	4	9	14	33	50	21
19	Cesena	27	4	9	14	25	48	21
20	Parma	25	3	3	19	21	51	9

Una definición formal de matriz es la siguiente:

Definición: Sean m y n enteros positivos. Un arreglo rectangular en m filas y n columnas de números reales:

$$A = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{pmatrix}$$

Se llama **matriz** de orden (o tamaño) $m \times n$ sobre $\mathbb{R}$. Donde " m " es la cantidad de **filas** y " n " la cantidad de **columnas** que tiene la matriz.



(Es como una R doble)

$\mathbb{R}$ es una notación que significa "números reales" (ver este punto de esta Unidad)

En forma abreviada se escribe $A = (a_{ij})_{m \times n}$ donde a_{ij} denota la entrada de A en la fila i y la columna j .

El conjunto de las **matrices** de orden $m \times n$ con entradas reales se denota con **$M(m, n, \mathbb{R})$** . En el caso de **matrices cuadradas**, es decir con igual número de filas y columnas, se usa la notación **$M(n, \mathbb{R})$** y se dice que son de orden n .

Ejemplo:

Si $A \in M(2, 3, \mathbb{R})$ es la matriz de dos filas y tres columnas con entradas reales:

$$A = \begin{pmatrix} A & B & C \\ D & E & F \end{pmatrix}$$

Entonces $a_{12} = B$ y $a_{23} = F$, ya que:

a_{12} hace referencia a la posición de la matriz dada por:

- a = la matriz
- 1 = la fila
- 2 = la columna

Por lo tanto, la a_{12} nos va a permitir acceder al elemento que está en la fila 1 y columna 2, de la matriz “a”, cuyo valor es “B”.

ANEXO 1 - PREGUNTAS FRECUENTES

Si alguna vez escuchaste sobre la "alineación" de la memoria, ¡es esto mismo!

1) P: Si busco el 9B4 en binario, se obtiene 1001 1011 100, pero el ejemplo de los materiales incluye un 0 adicional. ¿Porqué?

*Si se buscara obtener bytes, tendríamos
1001 1011 0100 0000*

R: Porque se ~~buscar obtener un byte (8 bits) para~~ que puedan ser interpretados por los circuitos digitales actuales, por lo tanto, se separa cada número convertido a binario de 4 en 4:

9 = 1001

B = 1011

4 = 100

Pero a este último le falta un dígito, por lo tanto, se incorpora un cero a la izquierda para completar los 4 dígitos. Como consecuencia, el resultado es 1001 1011 0100

rellenar con ceros a la izquierda

2) P: ¿El ampersand (&), significa conjunción "^"?

R: Correcto: se puede usar "^", "&" o "Y".

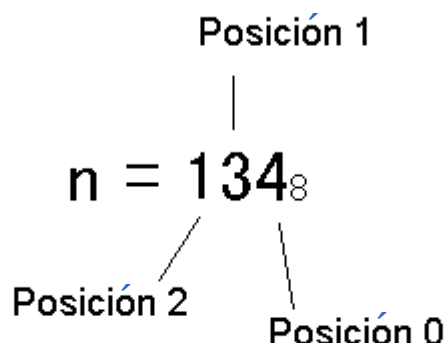
3) P: ¿Qué significa que $92_{10} = 134_8$?

R: Que son valores equivalentes (el mismo) pero expresados de otra manera. ¿Cómo llegamos de uno a otro? Así:

Empecemos con entender que 134_8 significa "134 en octal" o "134 en base 8".

Como ya se mencionó antes, las diferentes bases mencionadas son la decimal, hexadecimal, binaria y octal.

El número en octal tiene posiciones y se empieza a contar desde la derecha a la izquierda, desde el 0 por eso el 4 tiene posición 0, el 3 posición 1 y la 1 posición 2:



Para comprobar la equivalencia, se debe pasar ese número de una base a otra.

Para esto se toma el número y se lo multiplica por el sistema en el que se encuentra (en este caso el 8) y este sistema se eleva o potencia por la posición del número que tomamos.

Por lo tanto:

$$(1 * 8^2) + (3 * 8^1) + (4 * 8^0)$$

En el primer término de la expresión ("($1 * 8^2$)") el 1 se obtiene del número a convertir, el 8 es la base en la cual se encuentra expresado el número y la elevación a 2 es la posición.

Lo mismo con el resto de los términos.

Resolviendo:

$$(1 * 8^2) + (3 * 8^1) + (4 * 8^0)$$

$$(1 * 64) + (3 * 8) + (4 * 1)$$

$$64 + 24 + 4 = 92$$

Para indicar que el resultado está en la base 10, se puede agregar subíndice con la base:

92_{10}

Por lo tanto 134_8 es igual a 92_{10} .

4) P: ¿En programación, en qué momento se utilizan las conversiones de bases y me tengo que dar cuenta que es necesario convertir de una base a la otra?

R: No se utilizan explícitamente, pero es importante saber de dónde salen y, sobre todo, que existen. El binario no es algo que sale de un repollo, sino una forma de representar datos "humanos" ^{de} a una forma que sean interpretables por un mecanismo digital. Las direcciones de memoria, por ejemplo, se suelen expresar en hexadecimal, porque en binario serían números muy largos, y así. Cada cosa tiene un sentido y, ya veremos, algunas se aplican en forma directa en la programación y otras en forma más tangencial, pero todas tiene algún punto que les toca.

5) P: En la conversión de Hexadecimal a Binario, ¿de dónde salen los 4 dígitos?

R: Tomando como ejemplo la conversión de $BC5_{16}$ a binario

$B(11) = 1011$

$C(12) = 1100$

$5 = 0101$

$BC5_{16} = 1011\ 1100\ 0101_2$

Porque $\log_2(16) = 4$
(chequeá mi apunte en esa página)

Las conversiones que se hacen acá son:

$B(11) = 1011$

B en hexadecimal, que en decimal es 11, se puede expresar en binario como 1011, ya que 11 decimal es 1011 binario, según se explica en esta unidad.

Lo mismo para: C(12) = 1100

Y en este caso, la conversión es más directa:

5 = 0101

Por lo tanto, concatenando los 3 números decimales obtenidos, se puede decir que $BC5_{16} = 1011\ 1100\ 0101_2$

6) P: ¿El número decimal 5 en binario es 101 sin el 0 adelante o con el cero adelante? ¿O ese cero se le agrega para completar los 4 dígitos?

R: El 0 a la izquierda se usa para redondear bloques de 4 bits. Y 2 bloques de 4 bits forman un byte. Ese sería el sentido. Por lo tanto, el cero a la izquierda ES representativo cuando se están representando bits y bytes en binario.